

PythonDay4

March 17, 2026

0.0.1 Pandas

Pandas is a powerful Python library for data manipulation and analysis.

It provides:

- Flexible data structures for handling structured data
- Tools for data cleaning, transformation, and analysis
- Fast and efficient operations on labeled data

Why use Pandas? It allows you to work with structured data (like tables or spreadsheets) in an intuitive way, enabling fast data analysis, cleaning, and transformation with minimal code. It is widely used in data science, machine learning, and data analytics.

Key Features

- **Series and DataFrame** – Core data structures for 1D and 2D labeled data
- **Labeled data** – Each row and column has labels, making data easier to understand and manipulate
- **Data alignment** – Automatic alignment of data based on labels
- **Handling missing data** – Built-in tools to detect, remove, or fill missing values (**NaN**)
- **Data manipulation** – Filtering, grouping, aggregation, merging, and reshaping data
- **GroupBy operations** – Split, apply, and combine data efficiently
- **Time series support** – Powerful tools for working with dates and time-based data
- **File I/O** – Read/write data from CSV, Excel, SQL, JSON, etc.
- **Indexing and selection** – Powerful ways to select and slice data (`loc`, `iloc`)
- **Vectorized operations** – Fast computations without explicit loops

- **Integration** – Works seamlessly with NumPy, Matplotlib, SciPy, and ML libraries
- **Performance** – Built on top of NumPy for fast computation

```
[1]: #pip install numpy
```

```
[2]: #pip install pandas
```

```
[3]: import pandas as pd
import numpy as np
# Pandas Basic DataStructure
# 1.Series: 1D labeled array holding data of any type
s = pd.Series([1,3,5,np.nan,6,8])
s
```

```
[3]: 0    1.0
1    3.0
2    5.0
3    NaN
4    6.0
5    8.0
dtype: float64
```

```
[4]: import pandas as pd
# Pandas Basic DataStructure
# 1.Series: 1D labeled array holding data of any type
s = pd.Series([1,3,5,np.nan,6,8])
s
```

```
[4]: 0    1.0
1    3.0
2    5.0
3    NaN
4    6.0
5    8.0
dtype: float64
```

```
[5]: # 2.DataFrame: 2D data structure
df= pd.read_csv("bmw_global_sales_2018_2025.csv")
df
```

```
[5]:
```

	Year	Month	Region	Model	Units_Sold	Avg_Price_EUR	\
0	2018	1	Europe	3 Series	7822	47482	
1	2018	1	Europe	5 Series	10280	61685	
2	2018	1	Europe	X3	3105	58433	
3	2018	1	Europe	X5	7420	67955	
4	2018	1	Europe	X7	8474	92300	
...	...	...	...	...	...	...	

3067	2025	12	RestOfWorld	X5	9281	68198
3068	2025	12	RestOfWorld	X7	12785	91839
3069	2025	12	RestOfWorld	i4	4906	63437
3070	2025	12	RestOfWorld	iX	7871	73867
3071	2025	12	RestOfWorld	MINI	6732	41811

	Revenue_EUR	BEV_Share	Premium_Share	GDP_Growth	Fuel_Price_Index
0	371404204	0.011	19.12	3.50	1.00
1	634121800	0.019	19.12	3.50	1.00
2	181434465	0.022	19.12	3.50	1.00
3	504226100	0.021	19.12	3.50	1.00
4	782150200	0.035	19.12	3.50	1.00
...	...	...	...	...	...
3067	632945638	0.201	5.89	2.37	1.41
3068	1174161615	0.203	5.89	2.37	1.41
3069	311221922	0.180	5.89	2.37	1.41
3070	581407157	0.196	5.89	2.37	1.41
3071	281471652	0.167	5.89	2.37	1.41

[3072 rows x 11 columns]

```
[6]: new_df = df.iloc[:, :-6]
```

```
[7]: new_df
```

```
[7]:
```

	Year	Month	Region	Model	Units_Sold
0	2018	1	Europe	3 Series	7822
1	2018	1	Europe	5 Series	10280
2	2018	1	Europe	X3	3105
3	2018	1	Europe	X5	7420
4	2018	1	Europe	X7	8474
...	...	...	...	...	...
3067	2025	12	RestOfWorld	X5	9281
3068	2025	12	RestOfWorld	X7	12785
3069	2025	12	RestOfWorld	i4	4906
3070	2025	12	RestOfWorld	iX	7871
3071	2025	12	RestOfWorld	MINI	6732

[3072 rows x 5 columns]

```
[8]: # df
```

```
[9]: # demo_df = df.drop(df.columns[-6:], axis=1)
```

```
[10]: new_df.iloc[3]
```

```
[10]: Year          2018
      Month          1
      Region        Europe
      Model          X5
      Units_Sold     7420
      Name: 3, dtype: object
```

```
[15]: new_df.loc[5,"Region"]
```

```
[15]: 'Europe'
```

```
[17]: # Data cleaning
      new_df.isnull()
```

```
[17]:
```

	Year	Month	Region	Model	Units_Sold
0	False	False	False	False	False
1	False	False	False	False	False
2	False	False	False	False	False
3	False	False	False	False	False
4	False	False	False	False	False
...	...	...	...	...	...
3067	False	False	False	False	False
3068	False	False	False	False	False
3069	False	False	False	False	False
3070	False	False	False	False	False
3071	False	False	False	False	False

[3072 rows x 5 columns]

```
[19]: new_df.duplicated()
```

```
[19]: 0      False
      1      False
      2      False
      3      False
      4      False
      ...
      3067   False
      3068   False
      3069   False
      3070   False
      3071   False
      Length: 3072, dtype: bool
```

```
[20]: new_df.head()
```

```
[20]:
```

	Year	Month	Region	Model	Units_Sold
0	2018	1	Europe	3 Series	7822

1	2018	1	Europe	5 Series	10280
2	2018	1	Europe	X3	3105
3	2018	1	Europe	X5	7420
4	2018	1	Europe	X7	8474

```
[21]: new_df.drop_duplicates()
```

```
[21]:
```

	Year	Month	Region	Model	Units_Sold
0	2018	1	Europe	3 Series	7822
1	2018	1	Europe	5 Series	10280
2	2018	1	Europe	X3	3105
3	2018	1	Europe	X5	7420
4	2018	1	Europe	X7	8474
...	...	...	...	...	...
3067	2025	12	RestOfWorld	X5	9281
3068	2025	12	RestOfWorld	X7	12785
3069	2025	12	RestOfWorld	i4	4906
3070	2025	12	RestOfWorld	iX	7871
3071	2025	12	RestOfWorld	MINI	6732

[3072 rows x 5 columns]

```
[23]: # Rename function: df.rename(columns={old_name: new_name}, inplace=True)
# here inplace=True makes changes in original column
# df.rename(columns={
#     "A": "Age",
#     "B": "Salary",
#     "C": "Name"
# }, inplace=True)

# Rename index
# df.rename(index={0: "row1", 1: "row2"})
```

```
[31]: # Data Manipulation
#Column Operations:

new_df["Region_code"] =new_df["Region"].map({"Europe": 1,
                                             "RestOfWorld" :0})

new_df
```

```
[31]:
```

	Year	Month	Region	Model	Units_Sold	Region_code
0	2018	1	Europe	3 Series	7822	1.0
1	2018	1	Europe	5 Series	10280	1.0
2	2018	1	Europe	X3	3105	1.0
3	2018	1	Europe	X5	7420	1.0
4	2018	1	Europe	X7	8474	1.0

...	...	...	...	...	...	...
3067	2025	12	RestOfWorld	X5	9281	0.0
3068	2025	12	RestOfWorld	X7	12785	0.0
3069	2025	12	RestOfWorld	i4	4906	0.0
3070	2025	12	RestOfWorld	iX	7871	0.0
3071	2025	12	RestOfWorld	MINI	6732	0.0

[3072 rows x 6 columns]

```
[32]: # df.drop(["A", "B"] axis =1)

# df.insert(loc, column, values)
```

```
[35]: # Apply Functions:

# df.apply(), column/row wise operations
new_df["Units_Sold"] = new_df["Units_Sold"].apply(lambda x: x * 1.10)

# # df.map(), mapping values in one column
# new_df["Region_code"] = new_df["Region"].map({
#     "Europe": 1,
#     "RestOfWorld": 0
# })
# new_df = new_df.applymap(lambda x: x.lower() if isinstance(x, str) else x)
# #used to apply function to every cell
```

```
[41]: # Total Units Sold by Region
df.groupby("Region")["Units_Sold"].sum()

# # Average Units Sold by Model
# df.groupby("Model")["Units_Sold"].mean()

# # Count of records per Year
# df.groupby("Year")["Units_Sold"].count()
```

```
[41]: Region
China      6256750
Europe     6045176
RestOfWorld 6113872
USA        6099647
Name: Units_Sold, dtype: int64
```

```
[43]: #2. Multiple Aggregations (agg)

# Multiple stats for each Region
df.groupby("Region")["Units_Sold"].agg(["sum", "mean", "max", "min", "count"])
```

```
[43]:
```

	sum	mean	max	min	count
Region					
China	6256750	8146.809896	15914	2498	768
Europe	6045176	7871.322917	15877	2427	768
RestOfWorld	6113872	7960.770833	15418	2533	768
USA	6099647	7942.248698	15426	2379	768

```
[44]: # Group by multiple columns
df.groupby(["Region", "Model"])["Units_Sold"].agg(["sum", "mean"])
```

```
[44]:
```

Region	Model	sum	mean
China	3 Series	711402	7410.437500
	5 Series	813129	8470.093750
	MINI	735790	7664.479167
	X3	794213	8273.052083
	X5	816868	8509.041667
	X7	763185	7949.843750
	i4	765519	7974.156250
	iX	856644	8923.375000
Europe	3 Series	733992	7645.750000
	5 Series	742724	7736.708333
	MINI	735605	7662.552083
	X3	709038	7385.812500
	X5	744099	7751.031250
	X7	820143	8543.156250
	i4	800865	8342.343750
	iX	758710	7903.229167
RestOfWorld	3 Series	768488	8005.083333
	5 Series	746266	7773.604167
	MINI	762095	7938.489583
	X3	763639	7954.572917
	X5	745215	7762.656250
	X7	784423	8171.072917
	i4	779974	8124.729167
	iX	763772	7955.958333
USA	3 Series	792166	8251.729167
	5 Series	750405	7816.718750
	MINI	737715	7684.531250
	X3	758971	7905.947917
	X5	778952	8114.083333
	X7	744323	7753.364583
	i4	779329	8118.010417
	iX	757786	7893.604167

```
[45]: # 3. Using transform() ## same size of rows .. doesnot reduce rows
```

```
# Add column: total sales per Region
df["Region_Total"] = df.groupby("Region")["Units_Sold"].transform("sum")
```

```
[46]: # Add column: average sales per Model
df["Model_Avg"] = df.groupby("Model")["Units_Sold"].transform("mean")
```

```
[48]: # 4. Compare Individual vs Group

# contribution of each row to its Region
df["Percent_of_Region"] = (
    df["Units_Sold"] / df.groupby("Region")["Units_Sold"].transform("sum")
) * 100
```

```
[49]: #5. Custom Aggregation

df.groupby("Region")["Units_Sold"].agg(
    total_sales="sum",
    avg_sales="mean",
    max_sales="max"
)
```

```
[49]: Region
China          6256750
RestOfWorld    6113872
USA            6099647
Europe         6045176
Name: Units_Sold, dtype: int64
```

```
[50]: # 6. Group by Month & Year

df.groupby(["Year", "Month"])["Units_Sold"].sum()
```

```
[50]: Year  Month
2018   1      230195
       2      201033
       3      250912
       4      249552
       5      218362
       ...
2025   8      260474
       9      306776
      10      237864
      11      285728
      12      299002
Name: Units_Sold, Length: 96, dtype: int64
```

```
[51]: # 7. Reset Index

result = df.groupby("Region")["Units_Sold"].sum().reset_index()
```

```
[52]: # 8. Sort Grouped Data

df.groupby("Region")["Units_Sold"].sum().sort_values(ascending=False)
```

```
[52]: Region
China          6256750
RestOfWorld    6113872
USA            6099647
Europe         6045176
Name: Units_Sold, dtype: int64
```

```
[53]: #Merging
df1 = pd.DataFrame({
    'ID': [1, 2, 3],
    'Name': ['A', 'B', 'C']
})

df2 = pd.DataFrame({
    'ID': [2, 3, 4],
    'Salary': [50000, 60000, 70000]
})

result = pd.merge(df1, df2, on='ID', how='inner')
print(result)
```

```
   ID Name  Salary
0   2   B   50000
1   3   C   60000
```

Feature	merge()	concat()	join()
Purpose	Combine using common column	Stack/append data	Combine using index
How it works	SQL-style join	Axis-based stacking	Index-based merge
Key parameter	on="column"	axis=0 or 1	Uses index (default)
Join types	inner, left, right, outer	No join types	left, right, outer, inner

Operation	When to Use	Real-Life Example
merge()	When datasets share a common column (key)	Combine customers + orders using customer_id

Operation	When to Use	Real-Life Example
<code>concat()</code>	When you want to stack similar data	Combine January + February sales data
<code>join()</code>	When data is aligned by index	Add columns from another table with same row index

syntax

merge

```
pd.merge(df1, df2, on="column", how="type")
pd.merge(df1, df2, on="column", how="type"), type= left, outer, etc
```

concat

```
pd.concat([df1, df2]) # row wise
pd.concat([df1, df2], axis=1) # column wise
```

join

```
df1.join(df2, how= "Left")
```

[]: